

PHASE 3 OF 5

UI/UX Development & Customization

Metro Ticket Generation System — ServiceNow

Field	Detail
Phase Purpose	Convert the backend foundation into a usable passenger-facing Service Catalog experience.
Primary Activities	Interface design, catalog item, variables, navigation flow, Catalog Client Scripts, UI Policies and QR-ticket experience.
Main Catalog Item	Book A Metro Ticket
Expected Outcome	A dynamic booking form that guides users, calculates fare values, conditionally displays fields and presents a QR-ticket experience.

1. Phase 3 Overview

Phase 3 focuses on the user experience layer. The goal is to make the Metro Ticket Generation System easy to use while connecting the interface to the backend data and business logic created in Phase 2.

Backend Data → Service Catalog Item → Variables → Dynamic Client Logic → UI Policies → QR Widget → Checkout/Ticket Experience

2. UI/UX Design Principles

- Simple: Ask the passenger only for information required to generate the ticket.
- Guided: Use references, choices and conditional fields so the form explains itself.
- Dynamic: Fare values should react to station, journey type and passenger count.
- Safe: Invalid or incomplete input should be blocked before submission.
- Consistent: Use clear labels such as Starting From, Going To, Type of Journey and No. of Passengers.
- Responsive: The catalog experience should work cleanly in the intended ServiceNow interface.

3. Navigation Flow

Application Navigator → Service Catalog → Maintain Items → Book A Metro Ticket → Variables / Client Scripts / UI Policies → Test Catalog Form

4. Phase 3 Brainstorming

- Which fields should be visible immediately and which should appear only when needed?
- Should Starting From and Going To reference metro station records?
- Should fare fields be read-only so users cannot manually alter calculated amounts?
- What happens when passenger count changes after route selection?
- How should Single Journey and Return Journey affect displayed amount?
- When should the QR code be generated — before checkout, after confirmation, or as part of the final ticket?
- What should happen if payment mode is Others?

5. Activity 1 — Creation of Service Catalog Item

The primary UI entry point is a Service Catalog item named Book A Metro Ticket. It acts as the passenger booking form and hosts variables, UI policies and Catalog Client Scripts.

Procedure

- Open the Application Navigator.
- Click All and search for Service Catalog.
- Under Service Catalog, navigate to Maintain Items.
- Click New.
- Enter the item name as Book A Metro Ticket.
- Select Catalog → Service Catalog.
- Select the Services category.

- Enter a short description if required.
- Save the catalog item.

Suggested short description

“Book a metro journey by selecting source, destination, journey type, passenger count and payment details. Fare is calculated dynamically.”

6. Catalog Item Responsibilities

Component	Purpose
Catalog Item	Provides the user-facing booking entry point.
Variables	Capture journey and payment information.
Catalog Client Scripts	Perform dynamic client-side behavior such as fare calculation and submit handling.
Catalog UI Policies	Control visibility, mandatory state and behavior based on choices.
Widget	Provides a custom QR-ticket presentation when required.

7. UI Structure

Section	Fields
Journey Details	Starting From, Going To, Type of Journey, No. of Passengers
Fare Details	Amount for Single Journey, Amount Including Return
Payment	Mode of Payment, Enter Payment Mode
Ticket Output	QR-ticket presentation / confirmation experience

8. Activity 2 — Variables Configuration

Variables are the input controls displayed on the Book A Metro Ticket catalog form. Each variable should have a meaningful label, appropriate type, internal name and validation behavior.

Variable configuration procedure

- Open the catalog item.
- Open the Variables related list and click New.
- Select reference, multiple choice, select box or single-line text as appropriate.
- Enter the variable question/label and stable internal name for scripting.
- Configure choices or reference table where required.
- Add tooltip/example text when useful.
- Set mandatory/read-only behavior according to the requirement.
- Save and test the variable on the catalog form.

9. Variables to Create

#	Variable / Question	Type	Choices / Reference
1	Starting From	Reference	u_metro_station_details
2	Going To	Reference	u_metro_station_details
3	Type of Journey	Multiple Choice	Single Journey (1), Return Journey (2)
4	No. of Passengers	Select Box	1, 2, 3, 4
5	Amount for Single Journey	Single Line Text	Calculated fare
6	Amount Including Return	Single Line Text	Calculated return fare
7	Mode of Payment	Multiple Choice	UPI, Card, Others
8	Enter Payment Mode	Single Line Text	Shown when Others is selected

10. Variable Names Used by Scripts

Display Label	Internal Name
Starting From	starting_from
Going To	going_to
Type of Journey	type_of_journey
No. of Passengers	no_of_passengers
Amount for Single Journey	amount_for_single_journey
Amount Including Return	amount_including_return
Mode of Payment	mode_of_payment
Enter Payment Mode	enter_payment_mode

Important: The exact variable names in the catalog item must match the names referenced by client scripts.

11. Catalog Client Script — Dynamic Fare Calculation

A Catalog Client Script reacts to user input. Here, the onChange script on the passenger-count variable reads source, destination and journey type, then populates the appropriate fare field.

Configuration procedure

- Open Service Catalog → Maintain Items.
- Open Book A Metro Ticket.
- Open the item's related lists.
- Select Catalog Client Script → New.
- Set script name and UI Type = All.
- Select Type = onChange.
- Set the variable to No. of Passengers.
- Enter the fare-calculation script and save.
- Test route, journey type and passenger-count combinations.

12. Fare Rules Implemented

Route	Single / Passenger	Return / Passenger
Ameerpet → Madhapur	₹100	₹200
Ameerpet → LB Nagar	₹100	₹200
Ameerpet → Kukatpally	₹100	₹200
Ameerpet → Jubilee Hills	₹100	₹200

Calculation rule: Total Fare = Fare Per Passenger × Number Of Passengers.

13. Client Script — Core Implementation

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    // 1. Get the number of passengers selected and convert to an integer number
    var passengers = parseInt(newValue, 10);
    if (isNaN(passengers)) {
        passengers = 1;
    }

    // 2. Fetch the current selected journey type
    var journeyType = g_form.getValue('type_of_journey');

    // 3. Define your base ticket prices (Change these if your college base prices are different)
    var baseSinglePrice = 100;
    var baseReturnPrice = 200;

    // 4. Calculate total amounts and update fields dynamically
    if (journeyType == '1') { // Single Journey
        var totalSingle = baseSinglePrice * passengers;
        g_form.setValue('amount_for_single_journey', totalSingle.toString());
    }
    else if (journeyType == '2') { // Return Journey
        var totalReturn = baseReturnPrice * passengers;
        g_form.setValue('amount_including_return', totalReturn.toString());
    }
}
```

Implementation note: Replace route placeholder IDs with the actual station sys_ids.

14. Catalog UI Policy — Conditional Fields

A Catalog UI Policy controls presentation or behavior based on conditions. Here it is used so that Enter Payment Mode is shown only when the user selects Others for Mode of Payment.

Scenario

Mode of Payment = Others → Enter Payment Mode becomes visible.

Procedure

- Open Catalog Items and select Book A Metro Ticket.
- Open the Catalog UI Policy related list.
- Click New.
- Set Applies to = Catalog Item.
- Select Book A Metro Ticket.
- Add condition: Mode of Payment is Others.
- Save the UI Policy.
- Open UI Policy Actions and click New.
- Select Enter Payment Mode.
- Set Visible = True.
- Update and test the catalog form.

15. Why Use a UI Policy?

- Improves usability by hiding irrelevant fields.
- Reduces user confusion and unnecessary input.
- Centralizes simple conditional UI behavior.
- Avoids custom JavaScript when standard UI Policy behavior is sufficient.

16. Catalog Client Script — QR Code Generation

A second Catalog Client Script is configured as an onSubmit script. It intercepts the first Checkout click, constructs a QR-code URL and opens the custom metro_qr_widget in a modal.

Checkout → onSubmit → Build Ticket URL → Build QR Image URL → Open spModal → metro_qr_widget → Continue checkout

Configuration

- Open Book A Metro Ticket → Related Lists → Catalog Client Script.
- Click New.
- Set it to apply to the catalog item.
- Select Type = onSubmit.
- Enter the QR modal logic and save.
- Test that the first submission opens the QR experience instead of immediately completing checkout.

17. QR Code Client Script

```
g_scratchpad.metroQrDone = g_scratchpad.metroQrDone || false;

// Validate that Source and Destination are not the same station
var source = g_form.getValue('starting_from');
var destination = g_form.getValue('going_to');
if (source && destination && source === destination) {
    g_form.addErrorMessage('Source and Destination stations cannot be the same. Please change your route.');
```

return false; // Blocks form submission

```
}

// Ensure amount field is populated based on selected type
var journeyType = g_form.getValue('type_of_journey');
var amountSingle = g_form.getValue('amount_for_single_journey');
var amountReturn = g_form.getValue('amount_including_return');
if (journeyType == '1' && (!amountSingle || amountSingle == '0')) {
    g_form.addErrorMessage('Fare calculation error. Single journey fare cannot be empty.');
```

return false;

```
}

if (journeyType == '2' && (!amountReturn || amountReturn == '0')) {
    g_form.addErrorMessage('Fare calculation error. Return journey fare cannot be empty.');
```

return false;

```
}

function onSubmit() {
    if (g_scratchpad.metroQrDone === true) {
        return true;
    }

    var sysId = g_form.getUniqueValue();
    var dummyURL = 'https://example.com/ticket?id=' + sysId;
    var qrImageUrl = 'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=' +
encodeURIComponent(dummyURL);

    // Embeds the image directly inside the message string without needing an external widget
    var htmlMessage = "<div style='text-align:center;'><img src='" + qrImageUrl + "' alt='Metro QR'></div>";

    spModal.open({
        title: "Your Metro QR Ticket – click Checkout once more to confirm",
        message: htmlMessage,
        buttons: [{label: 'OK', primary: true}]
    });
}
```

```

    }).then(function() {
        g_scratchpad.metroQrDone = true;
        g_form.submit();
    });
    return false;
}

```

Production note: The sample uses a placeholder ticket URL and an external QR image service. In production, the QR payload should point to a secure ticket-validation endpoint and should not expose sensitive information.

18. End-to-End UI/UX Flow

Step	User Action	System Behavior
1	Open Book A Metro Ticket	Display catalog form.
2	Select Starting From / Going To	Reference variables provide valid station choices.
3	Select Type of Journey	Single/Return option is captured.
4	Select No. of Passengers	onChange script evaluates route + journey type + count.
5	Review fare	Applicable fare is populated; non-applicable field is cleared.
6	Select payment mode	If Others is selected, Enter Payment Mode becomes visible.
7	Click Checkout	onSubmit opens QR widget and blocks first submission.
8	Confirm / Checkout again	User continues intended ticket submission flow.

19. Phase 3 Test Cases

Test	Scenario	Expected Result
TC01	Valid route + 1 passenger + Single	Correct single fare populated.
TC02	Valid route + 2 passengers + Return	Return fare = per-passenger return fare × 2.
TC03	Change passenger count	Fare recalculates dynamically.
TC04	Unsupported route	Fare fields are cleared.
TC05	Payment = Others	Enter Payment Mode becomes visible.
TC06	Payment = UPI/Card	Enter Payment Mode remains hidden.
TC07	First Checkout	QR modal opens and submission is blocked.
TC08	QR confirmation	User can continue checkout.
TC09	Missing journey input	Submission/processing is prevented per validation design.
TC10	Intended UI types	Catalog behavior is verified in the target interface(s).

20. Phase 3 Exit Criteria

- Catalog item is available and correctly categorized.
- All approved variables are created with correct types and internal names.
- Fare calculation works for every approved route/journey combination.
- UI Policy correctly controls Enter Payment Mode visibility.
- QR widget/modal behavior works as designed.
- Positive, negative and boundary test cases pass.
- Catalog experience is ready for deeper business logic, automation, notifications and production hardening.

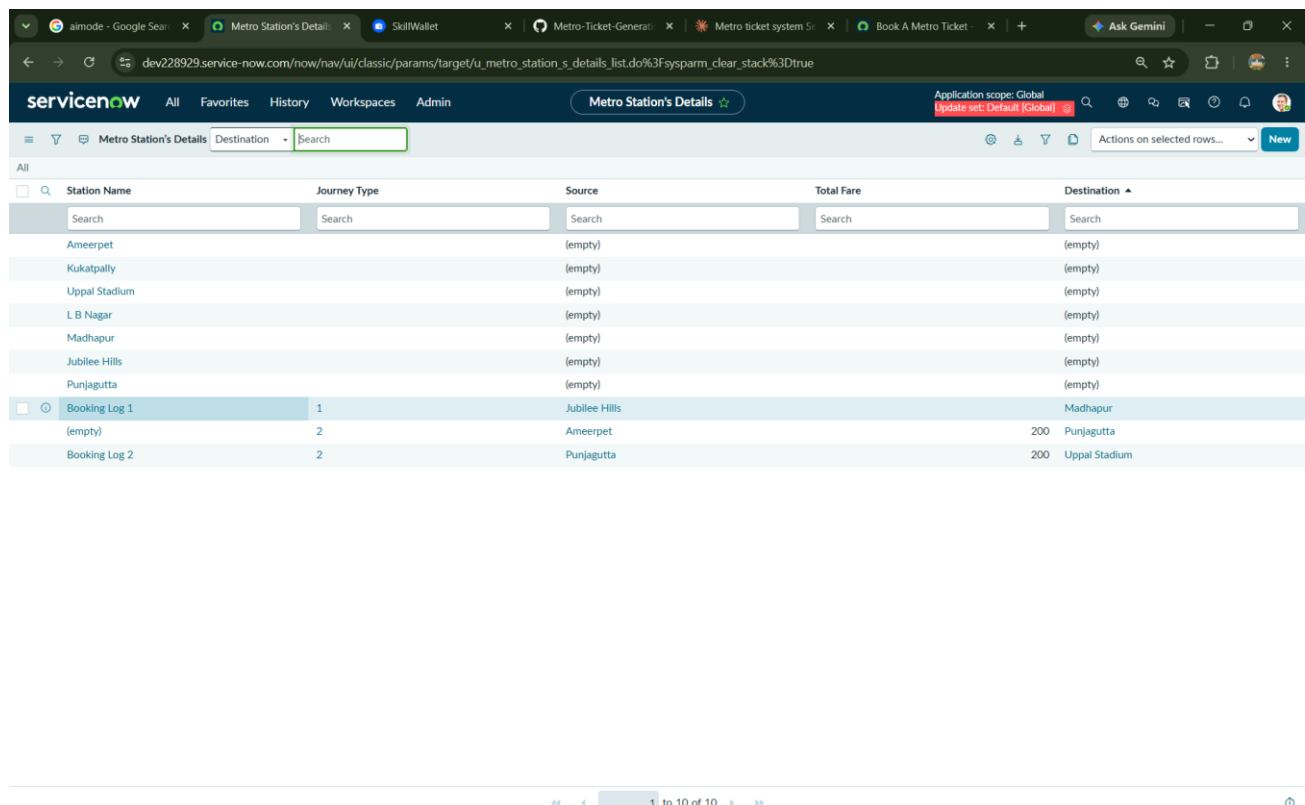
Final Outcome

Phase 3 transforms the Metro Ticket Generation System from a backend data model into a dynamic, user-friendly ServiceNow booking experience. The catalog item becomes the primary interface through which passengers provide journey details, receive calculated fare values and interact with the QR-ticket experience.

21. ServiceNow Implementation Evidence

The following screenshots correspond directly to the catalog item, variable configuration, passenger-facing catalog form, and QR widget activities described above.

Catalog Item Variables — Book A Metro Ticket



Station Name	Journey Type	Source	Total Fare	Destination
Search	Search	Search	Search	Search
Ameerpet		(empty)		(empty)
Kukatpally		(empty)		(empty)
Uppal Stadium		(empty)		(empty)
L B Nagar		(empty)		(empty)
Madhapur		(empty)		(empty)
Jubilee Hills		(empty)		(empty)
Punjagutta		(empty)		(empty)
Booking Log 1	1	Jubilee Hills		Madhapur
(empty)	2	Ameerpet	200	Punjagutta
Booking Log 2	2	Punjagutta	200	Uppal Stadium

Figure 1. Maintain Items view showing the eight configured variables.

Catalog Item Configuration — Book A Metro Ticket

The screenshot shows the ServiceNow Catalog Item form for 'Book A Metro Ticket'. The form is titled 'Catalog Item - Book A Metro Ticket' and includes a navigation bar with 'All', 'Favorites', 'History', 'Workspaces', and 'Admin'. The form fields are as follows:

- Name:** Book A Metro Ticket
- Application:** Global
- Active:** ☒
- Fulfillment automation level:** Unspecified
- Category:** Services
- State:** -- None --
- Checked out:** -- None --
- Owner:** System Administrator
- Secondary owners:** (empty)
- Short description:** A metro e-ticketing system
- Description:** (Rich text editor with a toolbar showing Bold, Italic, Underline, Link, etc.)

Figure 2. Catalog Item form showing name, category, active state, application scope and short description.

Service Portal Home — Passenger Entry Point

The screenshot shows the Service Portal landing page for 'QR generation'. The form is titled 'Catalog Client Scripts - QR generation' and includes a navigation bar with 'All', 'Favorites', 'History', 'Workspaces', and 'Admin'. The form fields are as follows:

- Name:** QR generation
- Applies to:** A Catalog Item
- Active:** ☒
- UI Type:** All
- Application:** Global
- Type:** onSubmit
- Catalog item:** Book A Metro Ticket
- Applies on a Catalog Item view:** ☒
- Applies on Requested Items:** ☒
- Applies on Catalog Tasks:** ☒
- Script:**

```

1 g_scratchpad.metroQrDone = g_scratchpad.metroQrDone || false;
2 // Validate that Source and Destination are not the same station
3 var source = g_form.getValue('starting_from');
4 var destination = g_form.getValue('going_to');
5
6 if (source && destination && source === destination) {
7   g_form.addErrorMessage('Source and Destination stations cannot be the same. Please change your route.');
```

Figure 3. Service Portal landing page providing the passenger-facing interface.

Book A Metro Ticket — Journey Details

How can we help?

Request Something
Browse the catalog for services and items you need

Knowledge Base
Browse and search for articles, rate or submit feedback

Get Help
Contact support to make a request, or report a problem

Current Status
No system is reporting an issue
More information...

My Assessments and Surveys
No assessments or surveys for you at the moment

My Approvals
You have no pending approvals

Announcements
Employee Center is available to you

Top Rated Articles
Getting Around in Windows
★★★★★

My Open Incidents
Employee payroll application server is down.
INC0007001 • 3mo ago
Rain is leaking on main DNS Server
INC0000016 • 3mo ago
JavaScript error on hiring page of corporate website
INC0000040 • 3mo ago
Can't launch 64-bit Windows 7 virtual machine
INC0000019 • 3mo ago
Sales forecast overview chart is DE&T F&M IV

Figure 4. Catalog form showing Starting From, Going To, journey type and passenger count.

Book A Metro Ticket — Fare and Payment Section

Home > Search

metro

Sources
All
Knowledge Bases
Catalogs

FILTERS
Knowledge Bases Clear All
Category Apply
Author
Last modified
View Count

Catalogs Clear All
Category Apply

All results for "metro"

Book A Metro Ticket
A metro e-ticketing system

Getting Around in Windows
of the Metro interface. The intent is to provide a seamless experience on desktops, laptops, tablets a Windows 8.x version and a pre-Windows 8 version. Windows 8.x applications (formerly called "metro").
Article: KB0000032 - Published: 11y ago

End of results

Figure 5. Lower portion of the catalog form showing fare outputs, Enter Payment Mode and checkout controls.

Metro QR Widget Configuration

Figure 6. Widget record for Metro QR Widget, including widget ID and Body HTML template.